

CSC9008 Cyber-Physical Systems

Shake it off: Final Presentation

Course Instructor: Prof. Chao Wang

Students: Xin-Shao Hon, Zhen-Rong Wu, Darrin Lin

Department of Computer Science and Information Engineering
National Taiwan Normal University

June. 2, 2026

Agenda

- 1 System Architecture
- 2 Recall: Basic background
- 3 Complementary Filter for Attitude Estimation
- 4 PID Control System
- 5 String-Length Geometry for MG90S Servos
- 6 System analysis
- 7 Live Demo

Project Overview

Goal

Build a self-stabilizing platform that keeps objects level despite external disturbances (manual tilting, vibration, earthquakes).

Hardware:

- STM32F401CCU6 (MCU)
- GY-521 / MPU6050 (IMU)
- 4 × MG90S servos
- Pulleys + strings
- Central universal joint

Control flow:

- Sense tilt via IMU
- Compute attitude (pitch, roll)
- PID computes correction
- Servos pull strings
- Platform tilts back to level

Hardware Price

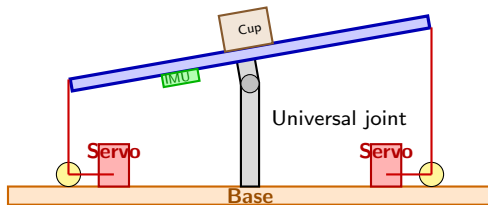
Component	Unit Price (NTD)	Quantity	Subtotal (NTD)
STM32F401CCU6	\$400	1	\$400
GY-521 / MPU6050	\$95	1	\$95
MG90S Servo	\$50	4	\$200
Clipboard	\$29	1	\$29
Pulley	\$19	4	\$76
String	\$14	1	\$14
Universal Joint	\$76	1	\$76
Socket	\$23	1	\$23
Screw Eye	\$12	1 pack	\$12
Serrated Flange Nut	\$20	1 pack	\$20
Washer	\$10	1 pack	\$10
Corrugated Board	\$12	1	\$12
Acrylic Sheet	\$140	1	\$140
Dupont Wires	\$20	lots	\$20
Arduino Mega	(\$400)	1	(\$400)
ST-link V2	(\$200)	1	(\$200)
Total			\$1127(1727)

Quick Demo

Quick Live Demo

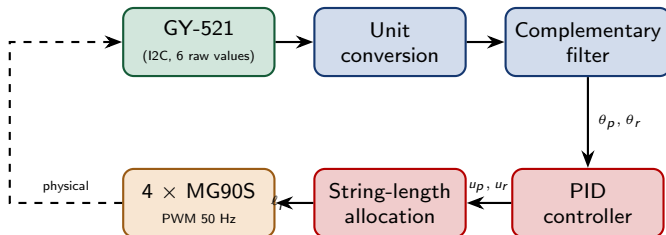
(We will have a more detailed demo at the end of the presentation)

Mechanical Structure



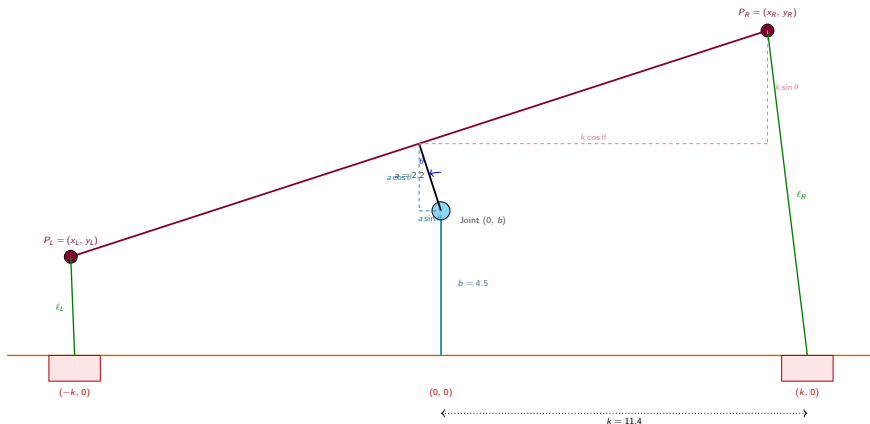
- 4 strings pulling 4 platform corners (PA0 back, PA1 left, PA2 right, PA3 front)
- Cross-pairs work antagonistically: one pulls in, the other releases
- Central universal joint bears the load; servos only adjust angle

Signal Flow Diagram



- Closed-loop at **500 Hz** (2 ms control period)
- Pitch and roll axes controlled independently
- All computation runs on STM32F401, USB-CDC for logging

Geometric Setup of the Mechanism



Platform tilts about the universal joint at $(0, b)$. The *upper pillar* (length a) rigidly connects the joint to the platform center, so it tilts with the platform. Strings run from platform ends to fixed servos at $(\pm k, 0)$.

Servo Action

Corrections u_p , u_r in **degrees of attitude correction**.

But each servo controls the **length of a string** that pulls the platform.

Naive approach (does not work well)

Map servo angle directly to angle output: $\theta_{\text{servo}} = u$.

Problem: 1° of servo rotation does *not* produce 1° of platform tilt. The relationship is highly nonlinear because strings connect a rotating platform to fixed servo positions.

Our approach

Compute the *exact string length* the platform would need at the desired tilt angle, then convert that length into a servo rotation via the spool radius.

Deriving the String Length (Step 1: Locate Platform Center)

The upper pillar rotates rigidly with the platform about the joint at $(0, b)$. The platform *center* is the top of the upper pillar. Before rotation it sits at $(0, b + a)$. The offset from the joint is $(0, a)$. After rotation by θ :

$$\text{offset}' = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} 0 \\ a \end{bmatrix} = \begin{bmatrix} -a \sin \theta \\ a \cos \theta \end{bmatrix}$$

Add back the joint position $(0, b)$:

$$P_{\text{center}} = (-a \sin \theta, b + a \cos \theta)$$

This term is what makes our model different from a naive single-pillar setup. Because a is not negligible compared to k , ignoring it would introduce a position error of up to $a \sin \theta_{\max}$ in the platform center.

Deriving the String Length (Step 2: Platform Endpoints)

Each platform endpoint is offset $\pm k$ along the platform direction. After rotation, the offsets become $(\pm k \cos \theta, \pm k \sin \theta)$.

Add to the platform center P_{center} :

$$P_R = (-a \sin \theta + k \cos \theta, b + a \cos \theta + k \sin \theta)$$

$$P_L = (-a \sin \theta - k \cos \theta, b + a \cos \theta - k \sin \theta)$$

String length is the Euclidean distance from each endpoint to its servo:

$$\ell_R = \|P_R - (k, 0)\|, \quad \ell_L = \|P_L - (-k, 0)\|$$

Result

$$\ell_R(\theta) = \sqrt{(-a \sin \theta + k \cos \theta - k)^2 + (b + a \cos \theta + k \sin \theta)^2}$$

$$\ell_L(\theta) = \sqrt{(-a \sin \theta - k \cos \theta + k)^2 + (b + a \cos \theta - k \sin \theta)^2}$$

Sanity Check at $\theta = 0$

Substituting $\sin 0 = 0$, $\cos 0 = 1$:

$$\ell_R(0) = \sqrt{(0 + k - k)^2 + (b + a + 0)^2} = a + b$$

$$\ell_L(0) = \sqrt{(0 - k + k)^2 + (b + a - 0)^2} = a + b$$

Both strings have length $a + b$ at rest ✓

String length *change* relative to neutral position:

$$\Delta\ell_R(\theta) = \ell_R(\theta) - (a + b), \quad \Delta\ell_L(\theta) = \ell_L(\theta) - (a + b)$$

Numerical example: $a = 2.2$, $b = 4.5$, $k = 11.4$, $\theta = 10^\circ$

$$\ell_R(10^\circ) \approx 8.66 \text{ cm}, \quad \Delta\ell_R \approx +1.96 \text{ cm (release)}$$

$$\ell_L(10^\circ) \approx 4.69 \text{ cm}, \quad \Delta\ell_L \approx -2.01 \text{ cm (reel in)}$$

The two strings move *antagonistically* (one shortens, one lengthens) — exactly the behavior we want.

From String Length to Servo Angle

Each servo has a spool with circumference C . One full 360° revolution winds/unwinds one full circumference of string. Therefore:

$$\Delta\ell = C \cdot \frac{\alpha}{360^\circ} \quad \Rightarrow \quad \boxed{\alpha = \frac{360^\circ \cdot \Delta\ell}{C}}$$

where α is the servo rotation (in degrees) and $\Delta\ell$ is the required string-length change.

Final servo command (relative to the 90° neutral position):

$$\theta_{\text{servo}} = 90^\circ + \alpha$$

Continuing the example ($\Delta\ell_R = +1.96$ cm, $C = 2\pi r$ with $r = 1.0$ cm
 $\Rightarrow C \approx 6.28$ cm)

$$\alpha_R = \frac{360 \cdot 1.96}{6.28} \approx 112^\circ$$

A 112° servo rotation is required to release 1.96 cm of string — which is why bigger spool circumference gives finer (but smaller-range) control.

What the MPU6050 Measures

Two sensors in one chip

Accelerometer: measures *specific force* (in g)

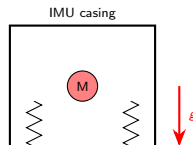
Gyroscope: measures *angular rate* (in deg/s)

Accelerometer (counter-intuitive!)

Even when stationary, the chip reads 1 g on z because:

$$\vec{a}_{\text{meas}} = \vec{a}_{\text{motion}} - \vec{g}$$

The spring inside resists gravity $\Rightarrow$ measurement is the reaction force.



Fuse Two Sensors

	Accelerometer	Gyroscope
Steady state	Accurate (gravity)	Slow drift
Fast motion	Corrupted by inertia	Accurate
Long term	No drift	Drift grows with t
Good in	Low frequency	High frequency

Key Idea

Use accelerometer for low-frequency reference (gravity is reliable)

Use gyroscope for high-frequency dynamics (no motion artefacts)

Blend them with a **frequency-domain crossover** $\Rightarrow$ Complementary Filter

Kalman filter would adapt the weights dynamically, but the fixed-weight complementary filter is sufficient for our low-speed platform and is far cheaper computationally (1 multiply + 1 add per axis).

The Complementary Filter

Discrete recursion

$$\hat{\theta}_k = \alpha \left(\hat{\theta}_{k-1} + \omega_k \Delta t \right) + (1 - \alpha) \theta_k^{\text{acc}}$$

- $\alpha \approx 0.96$: trust gyroscope short-term, use accelerometer to slowly correct drift.
- Equivalent to: **high-pass on gyro + low-pass on accel**, summing to unity gain.

Crossover frequency (Fourier interpretation):

$$f_c = \frac{1 - \alpha}{2\pi \alpha \Delta t} = \frac{0.02}{2\pi \cdot 0.96 \cdot 0.005} \approx 0.65 \text{ Hz}$$

Below 0.65 Hz: accel dominates (gravity reference); above: gyro dominates (fast motion).

Control Problem Statement

Objective

Drive the platform attitude (θ_p, θ_r) to $(0, 0)$ in the presence of disturbances.

Define the error signal:

$$e(t) = \theta_{\text{ref}}(t) - \theta(t) = 0 - \theta(t) = -\theta(t)$$

We want a control law $u(t)$ (servo correction in degrees) that:

- Reacts quickly to disturbances
- Eliminates steady-state error (when the table itself is tilted)
- Avoids overshoot and oscillation

⇒ **PID controller**, the workhorse of feedback control (Week 11).

The Three Terms of PID

Continuous form

$$u(t) = \underbrace{K_p e(t)}_{\text{Proportional}} + \underbrace{K_i \int_0^t e(\tau) d\tau}_{\text{Integral}} + \underbrace{K_d \frac{de(t)}{dt}}_{\text{Derivative}}$$

Term	Reacts to	Cures	Side effect
P	Current error	Slow response	Steady-state error
I	Accumulated error	Steady-state error	Overshoot, slow
D	Rate of change of error	Overshoot, damping	Noise amplification

Analogy:

P = spring (pulls toward target), I = memory (accumulates over time),

D = damper (resists motion)

Discrete Implementation

Approximate the integral by rectangular sum, derivative by backward difference:

$$\int_0^{t_k} e \, d\tau \approx \sum_{i=0}^k e_i \Delta t, \quad \frac{de}{dt} \approx \frac{e_k - e_{k-1}}{\Delta t}$$

Discrete PID

$$u_k = K_p e_k + K_i \cdot \Delta t \sum_{i=0}^k e_i + K_d \cdot \frac{e_k - e_{k-1}}{\Delta t}$$

Two practical safeguards:

- **Anti-windup:** clip the integral so it cannot grow unboundedly during saturation
- **Output saturation:** clip u to the servo's physical range

Discrete Velocity Form: PI Implementation

Standard Velocity Form PID: Subtracting positional u_{k-1} from u_k yields the required *change* Δu_k :

$$\Delta u_k = u_k - u_{k-1} = K_p[e_k - e_{k-1}] + K_i e_k + K_d[e_k - 2e_{k-1} + e_{k-2}]$$

Our Velocity Form PI Implementation

To avoid noise amplification from the second difference (D term), our firmware omits the K_d term and implements a **Velocity Form PI** controller:

$$\Delta u_k = K_{P_vel} [e_k - e_{k-1}] + K_{I_vel} e_k$$

Engineering Insight: In microcontrollers, the second difference $[e_k - 2e_{k-1} + e_{k-2}]$ is highly sensitive to sensor noise. Dropping it ensures a smoother control signal.

Velocity Form PI: Mathematical Equivalence

Why does it work? (When Accumulated)

Accumulating (integrating) the velocity step Δu_k over time mathematically reconstructs the standard positional control law:

$$u_k = \sum_{j=0}^k \Delta u_j = K_{P_vel} e_k + K_{I_vel} \sum_{j=0}^k e_j$$

- **K_{P_vel} acts as Proportional (P):**

Multiplies the first difference $[e_k - e_{k-1}]$. When accumulated, the intermediate terms cancel out (telescoping sum), leaving just the current error e_k .

- **K_{I_vel} acts as Integral (I):**

Multiplies the current error e_k directly. When summed over time, it naturally becomes the accumulated historical error ($\sum e_j$).

Full Control Pipeline

- 1 Read IMU, compute θ_p and θ_r via complementary filter.
- 2 Run two independent PIDs:

$$u_p = \text{PID}_p(0 - \theta_p), \quad u_r = \text{PID}_r(0 - \theta_r)$$

- 3 For each axis, compute the required string lengths using the geometric formulas:

$$\ell_R = \ell_R(u_p), \quad \ell_L = \ell_L(u_p)$$

- 4 Convert each string length to a servo rotation:

$$\alpha_i = \frac{360^\circ \cdot (\ell_i - (a + b))}{C}$$

- 5 Output PWM via TIM2 channels.

Same procedure runs for the roll axis.

All computations complete within one 5 ms control period.